

Kampus

Implementation Plan — MVP to Full Live Production

Technical roadmap for backend, database, real-time chat, auth, and QR-based stationery module

1. Architecture Overview

The platform moves from an in-memory / localStorage demo to a production-grade three-layer architecture: a React + Vite frontend, a Node.js + Express backend with a Socket.io real-time gateway, and a persistent cloud database with media storage.

Layer	Components	Responsibility
Frontend	React 18 + Vite, Tailwind UI, Axios/Fetch client, Socket.io client	UI rendering, API calls, live chat/notification updates
Backend	Express server, Socket.io gateway, JWT + college-domain auth middleware, REST routes	Business logic, auth, real-time events, request handling
Data	Supabase (PostgreSQL) + Supabase Storage	Persistent structured data, auth, file/image storage, real-time subscriptions

2. Database Decision — Finalized

Chosen: Option B — Supabase (PostgreSQL), not embedded SQLite/JSON.

Reasoning: the app's data (users, listings, chats, deals, ratings, stationery orders) is relational by nature, which fits PostgreSQL naturally. An embedded/file-based database risks data loss on serverless redeployments (Vercel/Render) and cannot handle concurrent access once real students are using the platform simultaneously. Supabase additionally bundles authentication, file storage, and real-time subscriptions in one service, reducing the number of moving parts the team has to maintain separately from Socket.io.

3. Phased Implementation Plan

Phase 1 — Real-Time Communication (Socket.io)

- Install **socket.io** on the backend and **socket.io-client** on the frontend.
- Implement core socket events: **join_chat** (private buyer/seller room), **send_message**, **send_offer** / counter-offer sync, **deal_confirmed**, **user_typing**, **online_status**.
- Add a global floating toast notification for new messages/offers arriving in any open tab.

Phase 2 — Persistent Database & Full API Integration

- Migrate **server/data/** from in-memory arrays to **Supabase PostgreSQL** tables so data survives restarts.
- Transition all frontend contexts (AuthContext, ListingsContext, ChatContext, StoreContext) to consume live backend endpoints instead of localStorage snapshots.
- Implement robust error handling, loading states, and offline/network fallback UI.

Phase 3 — Media Upload, Payments & Anti-Spam

- Add **multer** (or Supabase Storage direct upload) on the backend (**/api/upload**) for item photos.
- Enable client-side image picker with crop/preview and multi-photo gallery support for listings.
- **Integrate a payment gateway (Razorpay recommended for India)** for stationery pre-orders, so payment is captured before a QR pickup code is issued.
- **Add basic rate-limiting and spam protection** (e.g. per-user listing/message limits, CAPTCHA on signup) to prevent fake listings and message spam once the platform is public.

Phase 4 — Production Auth & College Verification

- Standardize JWT authentication with secure password hashing or real OTP login (via Supabase Auth).
- Official campus email validation (**.edu, .ac.in**) with an automated "Verified Student" badge.
- Role-based access control: Student vs Store Merchant vs Campus Admin.

Phase 5 — Campus Store & Digital QR Verification

- Dynamic QR code generation for student stationery orders with a unique, non-guessable payload.
- Real-time merchant/counter dashboard scanner to instantly validate and fulfill orders on pickup.
- Confirm the module is branded under the platform's own name — not a placeholder college club name — before this ships to any real store counter.

4. Naming Consistency Fix

The original generated roadmap referred to a "Tanish SAC Store" module in Phase 5 — this appears to be a placeholder inserted by the build tool and does not match the platform's actual name. Before Phase 5 ships, rename all references (code, UI copy, and any merchant-facing materials) to the platform's finalized name so there is no mismatch between the app branding and the in-store QR verification screen.

5. Verification & Testing Plan

Automated / API Verification

- Test all REST endpoints (**/api/auth**, **/api/listings**, **/api/chats**, **/api/deals**, **/api/store**) via direct HTTP requests.
- Verify real-time message exchange between two distinct browser sessions over Socket.io.
- Verify payment webhook handling and QR code generation only fire after a confirmed successful payment.

Manual Verification

- Open two different browser tabs (e.g. User A in Tab 1, User B in Tab 2).
- Post an item in Tab 1 → confirm it appears instantly in Tab 2's feed.
- Start a chat in Tab 2 and make an offer → confirm Tab 1 receives a live notification and can Accept/Counter in real time.
- Both users confirm the deal → confirm listing status updates to "Sold/Rented" and the rating modal opens.
- Place a stationery test order → confirm QR code is only generated after payment succeeds, and that the merchant scanner correctly marks the order fulfilled.

6. Deployment Target

Frontend: deploy to **Vercel**. Backend: deploy to **Render** or **Railway**. Database/Auth/Storage: **Supabase** (managed, no separate server to maintain). This combination supports a 1-command local development launch as well as a straightforward cloud deployment path with no infrastructure to self-host.

7. Summary of Decisions Locked In

Decision point	Final choice	Reason
Database	Supabase (PostgreSQL)	Persistent, relational, bundles auth + storage + realtime
Payments	Razorpay integration in Phase 3	Needed before QR codes can be safely issued
Spam/abuse protection	Rate limits + signup CAPTCHA in Phase 3	Not covered in original roadmap
Module naming	Rename "Tanish SAC Store" to final platform name	Placeholder name from build tool

Decision point	Final choice	Reason
Deployment	Vercel (frontend) + Render/Railway (backend) + Supabase	Zero self-hosted infrastructure